With these challenges in mind, the alternative approach to CDCT is the provider-driven contract testing (PDCT) approach, also known as bi-directional, which is unique to PactFlow and not available in the open source Pact tool. In this penultimate chapter, we discuss in more detail why PDCT was introduced and how it differs from the CDCT approach we've been discussing so far in this book. We also share an example of how PDCT works in action. Finally, we provide an overview of PactFlow adapters that you can use to support PDCT and also walk you through some technical examples.

11.1 Introduction to provider-driven contract testing

Bi-directional contract testing, as defined by PactFlow in https://mng.bz/PNq9, is a type of static contract testing where two contracts—one representing consumer expectations, and another representing the provider's capability—are compared to ensure they are compatible. We describe this as *provider-driven* in this chapter because this is most likely the scenario where the OpenAPI contracts already exist. The PDCT approach comprises the following steps:

1. The provider uploads their version of a contract, which could be generated from an existing OpenAPI specification.
2. The consumer uploads their version of a contract, which is generated from existing tests.
3. The two contracts are uploaded to a PactFlow. PactFlow compares the two contracts and checks if they are compatible, a process known as cross-contract validation. PactFlow compares the shape of the consumer's expected response to the shape of the provider's actual response.
4. The provider or consumer uses Pact's can-i-deploy tool to check if they are safe to deploy their changes to production.
5. The provider or consumer deploys their changes.

The steps are also visualized in figure 11.2, which you've seen already in chapter 3, section 3.7.2.